

EKSAMENSOPPGAVE

Bokmål

Emnekode: DAT108

Emnenavn: Programmering og webapplikasjoner

Klasse: 2. klasse DATA / INF

Dato: 19. juni 2025

Eksamensform: Skriftlig skoleeksamen (Wiseflow | FLOWmulti)

Eksamenstid: 4 timer (0900-1300)

Antall eksamensoppgaver: 6

Antall sider (medregnet denne): 10

Antall vedlegg: Ingen

Tillatte hjelpemiddel: Ingen

Lærere: Lars-Petter Helland (928 28 046) Bergen
 Bjarte Wang-Kileng (909 97 348) Bergen
 Erlend Raa Vågset (57 72 26 08) Førde
 Tord Waage Frøland (...) Haugesund

LYKKE TIL!

Oppgave 1 (10% ~ 24 minutter) – Strømmer

Flervalgsoppgave: Oppgavetekst i Wiseflow

Oppgave 2 (10% ~ 24 minutter) – Funksjonelle kontrakter

Flervalgsoppgave: Oppgavetekst i Wiseflow

Oppgave 3 (10% ~ 24 minutter) – JavaScript teori

Flervalgsoppgave: Oppgavetekst i Wiseflow

∞∞∞

Felles info for oppgave 4, 5 og 6

Du skal svare på disse i Wiseflow. Ved å bruke kodevedlegg vil dere få uthevet syntaks og korrekte innrykk. Vi anbefaler ett vedlegg per oppgave. For å sikre automatisk lagring, følg denne oppskriften.

- 1 - Start med å gi vedlegget et navn. For eksempel «Oppgave 4.java».
- 2 - Klikk lagre. Da aktiverer du automatisk lagring.
- 3 - Start å skrive kode

Pass på at du har svart på alle oppgaver før du prøver å få oppgaver perfekte. Hvis du mener noe er uklart i oppgaven, presiser forutsetningene dine i besvarelsen.

Oppgavene begynner på neste side, og er i tillegg delt ut på papir.

Oppgave 4 (15% ~ 36 minutter) – Tråder og trådsikkerhet

Du skal lage et program som oppretter to tråder. Hver tråd skal utføre en løkke som øker verdien til en felles tellevariabel et antall ganger. F.eks. hvis tellevariabelen starter på 0, og løkken har 10 000 gjennomløp, så skal/bør telleren ha verdien 20 000 når begge trådene er ferdige. Du kan bruke en `static int teller` som en «global» delt tellevariabel. Programmet skal skrive ut på skjermen den endelige verdien i telleren etter at begge trådene er ferdige.

Du kan velge selv hvordan du definerer trådklassen(e) og oppretter trådobjektene. Her er det flere alternativer, både via Thread og Runnable, og både via navngitt(e) og anonym(e) klassedefinisjon(er).

Først skal du lage en versjon som fører til en kappløpstilstand (race condition), og deretter en versjon der problemet er rettet opp. Hele oppgaven skal leveres i ett vedlegg, med tydelig merkede deloppgaver.

a) Kappløpstilstand (ca. 10 %)

Skriv en versjon av programmet beskrevet over, inkl. definisjon av den/de nødvendige trådklasser, der vi har en kappløpstilstand (race condition). Svar i tillegg på følgende spørsmål:

- i) Hva prøver programmet å gjøre? Forklar kort.
- ii) Hva kan gå galt, og hvorfor?
- iii) Hvor i koden skjer feilen(e)?

b) Rettet kode (ca. 5 %)

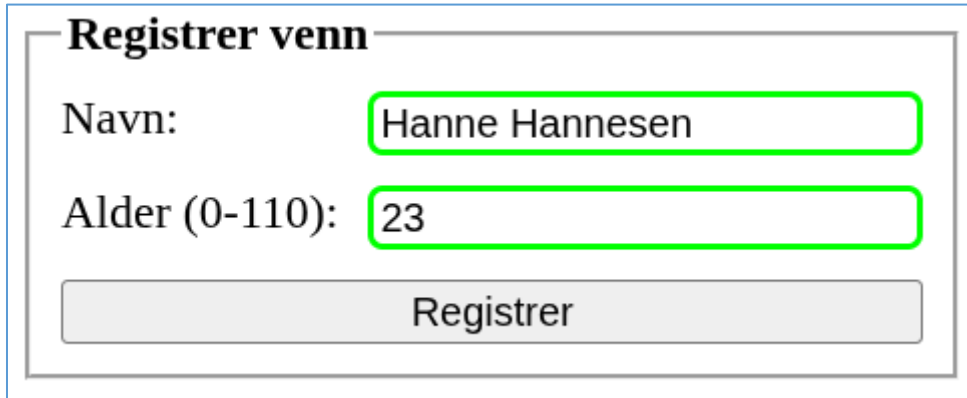
Skriv en versjon av programmet der svakheten(e)/feilen(e) er rettet ved hjelp av synkronisering, og svar i tillegg på følgende spørsmål:

- i) Hvilken løsning brukte du? Forklar kort.
- ii) Hvor i koden brukte du de(n)?
- iii) Hvorfor fungerer det?

Oppgave 5 (15% ~ 36 minutter) – JavaScript programmering

En webapplikasjon skal lages for å organisere en liste med venner. Det er ingen lagring av data på webtjener.

Websiden til applikasjonen inneholder to felt. Det første feltet lar bruker registrere en venn med navn og alder. Skjermbildet under viser hvordan dette feltet kan se ut:



Registrer venn

Navn:

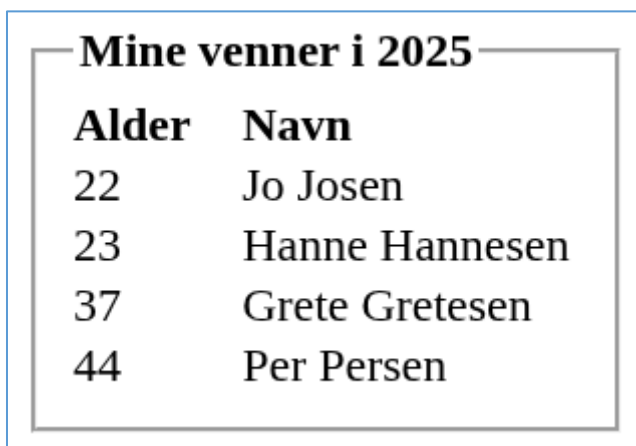
Alder (0-110):

Skjerm bilde 1: Felt for å registrere venn

Ingenting skal skje når bruker klikker *Registrer* knappen dersom:

- Et av feltene mangler verdi.
- Alder har en ulovlig verdi. **Alder** må være et heltall i intervallet 0 til 110.

Når knappen *Registrer* blir klikket skal vennen vises sortert på alder slik det neste skjermbildet, Skjerm bilde 2 viser:



Mine venner i 2025	
Alder	Navn
22	Jo Josen
23	Hanne Hannesen
37	Grete Gretesen
44	Per Persen

Skjerm bilde 2: Registrerte venner

Observer at listen med venner skal vises sortert på alder. Skjerm bilde 2 viser vennen fra Skjerm bilde 1 plassert riktig i listen. Årstallet som vises øverst, er året for dagen i dag.

Feltet i Skjerm bilde 1 for å registrere en venn er laget med HTML-koden som er vist i Kode eksempel 1 nedenfor:

```
<fieldset id="registrering">
  <legend>Registrer venn</legend>

  <label for="navn">Navn:</label>
  <input type="text" size="20" id="navn">
  <label for="alder">Alder (0-110):</label>
  <input type="number" id="alder" min="0" max="110">
  <button type="button">Registrer</button>
</fieldset>
```

Kode eksempel 1: HTML-kode for å registrere venn

Feltet i Skjerm bilde 2 for å vise listen med venner er laget med HTML-koden i Kode eksempel 2 nedenfor:

```
<fieldset id="visning">
  <legend>Mine venner i <span></span></legend>
  <table>
    <thead><tr><th>Alder</th><th>Navn</th></tr></thead>
    <tbody></tbody>
  </table>
</fieldset>
```

Kode eksempel 2: HTML-kode for å vise liste med venner

Applikasjonen inneholder følgende JavaScript-kode:

```
class Vennkontroller {
  #navnefelt;
  #alderfelt;
  // Fyll inn evt. flere felt

  constructor(felt1, felt2) {
    // Kode for felt1, felt for å registrere venn
    this.#navnefelt = felt1.getElementsByTagName("input")[0];
    this.#alderfelt = felt1.getElementsByTagName("input")[1];
    felt1.querySelector("button").addEventListener(
      "click",
      () => { this.#registrervenn() }
    );

    // Legg inn evt. mere kode her
  }

  #registrervenn() {
    /* Legg inn kode her for å vise venn fra Skjermbilde 1 i listen
       som vises i Skjermbilde 2 */
  }
}

const felt1 = document.getElementById("registrering");
const felt2 = document.getElementById("visning");

new Vennkontroller(felt1, felt2);
```

Kode eksempel 3: JavaScript-kode for å registrere og vise venner

Oppgave: Fyll inn den manglende koden i JavaScript-klassen **Vennkontroller**.

Hjelp:

- HTML input-elementer har i JavaScript en egenskap *value* som er nåværende verdi til feltet.
- Om JavaScript sitt **Number**:
 - Statisk metode *parseInt(string)* konverterer fra **String** til **Number**.
 - Statisk metode *isInteger(arg)* returnerer sann hvis argumentet er et heltall, ellers usann.
- Om JavaScript sin **Array**:
 - Statisk metode *from(liste)* returnerer forekomst av **Array** fra en listestruktur.

```
const HTMLTbl = document.getElementsByTagName("table")[0];
const rows = HTMLTbl.tBodies[0].rows;
const rowArray = Array.from(rows);
```

- Metode *findIndex(funksjon)* returnerer første indeks i forekomst av **Array** der test-funksjonen returnerer sann. Metoden returnerer verdi -1 hvis test-funksjonen er usann for alle tabellelementer.

```
const tabell = [1, 5, 12, 8];
const partall = (element) => element % 2 === 0;

console.log(tabell.findIndex(partall));
// Skriver ut tallet 2, indeks til 12 som er første partall
```

- Om JavaScript sitt **HTMLTableSectionElement** (HTML tagger *TBODY* og *THEAD*):
 - Metode *insertRow(indeks)* setter inn og returnerer en ny HTML tabellrekke (tagg *TR*) i posisjon *indeks* i elementet.

Dersom *indeks* har verdi -1 legges tabellrekken inn på slutten av elementet.
- Om JavaScript sitt **HTMLTableRowElement** (tagg *TR*):
 - Metode *insertCell(indeks)* setter inn og returnerer en ny HTML tabellcelle (tagg *TD*) i posisjon *indeks* i tabellrekken.

Dersom *indeks* har verdi -1 legges tabellcellen inn på slutten av tabellrekken.
- HTML **Node** elementet sin egenskap *textContent* representerer alt tekstinnhold i noden og alle dens barn.

Egenskapen kan tilordnes en ny verdi og vil da sette inn nytt tekstinnhold, f.eks. i en tabellcelle.

Oppgave 6 (40% ~ 96 minutter) – Web back-end med Spring MVC

I denne oppgaven skal du skrive litt kode (Spring og JSP) for en tenkt nettbutikk som selger buttons.

Nedenfor ser du nettsiden. Øverste del viser varene som selges i butikken, og nederste del viser varene brukeren har lagt i handlekurven i økten/sesjonen. Hvis handlekurven er tom, vises teksten «*Handlekurven er tom*» i stedet for radene med varer.

Virkemåte:

- URL for nettsiden er **/butikk**, og denne bygges opp fra dynamiske data for vareutvalg og handlekurv. Data om varer (**id**, navn, pris og bilde) hentes fra en database.
- Når bruker velger et antall av en vare og trykker «Legg i handlekurv» (Sti: **/handle**), blir det valgte antall av denne varen lagt til i handlekurven. Deretter blir siden vist på nytt med oppdatert handlekurv.
- Når bruker trykker søppelbøtte [] til høyre for vare i handlekurven (Sti: **/fjerne**), blir denne varen fjernet helt fra handlekurven. Deretter blir siden vist på nytt med oppdatert handlekurv.
- Når bruker trykker «Gå til kassen» kan vi gå til en tenkt sti kalt **/kasse**.

Våre populære buttons!



Give me a break

Pris: 99,-

Antall:



Hello, World!

Pris: 109,-

Antall:



I love Java

Pris: 89,-

Antall:

Din handlekurv

Varenavn	Stykkpris	Antall	Beløp	
Hello, World!	109,-	3	327,-	
I love Java	89,-	5	445,-	
Totalt			772,-	Gå til kassen

Til bruk i løsningen din tenker vi at vi har klassene **Vare**, **Ordrelinje** og **Handlekurv**.

- **Vare** representerer en vare fra vareutvalget i databasen.
- **Ordrelinje** (ofte også kalt varelinje) er et antall av en vare (f.eks. 3 stk. «Hello World!»-button). Denne har også en metode som gir beløp for et antall av en vare.
- **Handlekurv** lar oss legge til et antall av en vare, fjerne en ordrelinje, hente ut listen av ordrelinjer. Denne har også en metode som gir oss totalbeløp for alt i handlekurven.

Du kan anta at disse klassene finnes og virker helt fint.

```
@Entity
public class Vare {

    @Id private int id;
    String navn;
    int pris;
    String bildefil;

    //Gettere m.m. ...

}
```

```
public class Ordrelinje {

    private Vare vare;
    private int antall;

    //Gettere m.m. ...

    public int getBelop() {
        return vare.getPris() * antall;
    }

}
```

```
public class Handlekurv {

    public void leggTilEtAntallAvEnVare(Vare vare, int antall) {}
    public void fjernOrdrelinje(int vareid) {}
    public boolean isTom() {}
    public Collection<Ordrelinje> getOrdrelinjer() {}
    public int getTotal() {}

}
```

Du kan også anta at det er definert et interface **VareRepository** som lar deg hente ut varer fra databasen. I løsningen kan du bruke dette direkte i controlleren, dvs. det er ikke nødvendig å putte en service mellom controller og repository.

Krav til løsningen din:

- Løsningen må bruke Spring-rammeverket for controlleren
- Løsningen må bruke JSP, Expression Language (EL) og Standard Template Lib (JSTL) for viewet
- Løsningen må følge Post-Redirect-Get-prinsippet (PRG)

Oppgaver

- a) (10%) Skriv en Spring-controller **ButikkController** som behandler en forespørsel om å se nettsiden, altså **GET /butikk**. Hvis ingen handlekurv finnes i sesjonen, skal det opprettes en tom handlekurv for sesjonen. Varer hentes på nytt fra repositoret ved hver forespørsel. Viewet for denne siden er **butikk**.
- b) (15%) Skriv et JSP-view **butikk.jsp** som bygger opp nettsiden ut fra de dataene som er klargjort. Husk at nettsiden må inneholde HTML-kode for å utføre alle brukervalg som vist i illustrasjonen, dvs. legge et antall av en vare i handlekurv, fjerne ordrelinje for en vare, og gå til kassen.
- Tips:
- Nettsiden på bildet i oppgaveteksten bruker et tabelloppsett for å få det ryddig. Dette trenger ikke du å gjøre i din besvarelse! Det som er viktig er datainnhold og logisk struktur for å bygge en fungerende side.
 - Nettsiden har en «Legg i handlekurv»-knapp per vare og en søppelbøtte []-knapp per ordrelinje. En grei løsning her kan være å legge opp til flere <form>-elementer, og identifisere disse med vare-id.
 - Du kan anta at vare.bildefil gir en komplett filsti til bildet for varen.
- c) (5%) Utvid **ButikkController** med en metode som behandler en forespørsel om å legge et antall av en vare i handlekurven. Sti: **/handle**. Du kan anta at forespørselen kommer med gyldige data fra butikksiden.
- d) (5%) Utvid **ButikkController** med en metode som behandler en forespørsel om å fjerne en ordrelinje for en vare fra handlekurven. Sti: **/fjerne**. Du kan anta at forespørselen kommer med gyldige data fra butikksiden.
- e) (5%) Slik applikasjonen er nå, vil brukeren starte med tom handlekurv igjen hvis nettleseren lukkes og brukeren kommer tilbake til butikken neste dag. Beskriv hvordan vi kunne «beholdt» handlekurven til neste økt. Løsningen skal ikke kreve innlogging, og skal heller ikke lagre noe i databasen mellom øktene. For full score må du beskrive løsningen i detalj, men du trenger ikke å skrive Java-kode.

EKSAMENSOPPGÅVE

Nynorsk

Emnekode: DAT108

Emnenamn: Programmering og webapplikasjonar

Klasse: 2. klasse DATA / INF

Dato: 10. juni 2024

Eksamensform: Skriftleg skuleeksamen (Wiseflow | FLOWlock)

Eksamenstid: 4 timer (0900-1300)

Tal på eksamensoppgåver: 6

Tal på sider (medrekna denne): 10

Tal på vedlegg: Ingen

Tillate hjelpemiddel: Ingen

Lærarar: Lars-Petter Helland (928 28 046) Bergen
 Bjarte Wang-Kileng (909 97 348) Bergen
 Erlend Raa Vågset (57 72 26 08) Førde
 Tord Waage Frøland (...) Haugesund

LUKKE TIL!

Oppgave 1 (10% ~ 24 minutt) – Straumar

Fleirvalsoppgåve: Oppgåvetekst i Wiseflow

Oppgave 2 (10% ~ 24 minutt) – Funksjonelle kontraktar

Fleirvalsoppgåve: Oppgåvetekst i Wiseflow

Oppgave 3 (10% ~ 24 minutt) – JavaScript teori

Fleirvalsoppgåve: Oppgåvetekst i Wiseflow



Felles info for oppgave 4, 5 og 6

Du skal svare på desse i Wiseflow. Ved å bruke kodevedlegg vil de få utheva syntaks og korrekte innrykk. Vi tilrår eitt vedlegg per oppgåve. For å sikre automatisk lagring, følg denne oppskrifta.

- 1 - Start med å gi vedlegget eit namn. Til dømes «Oppgåve 4.java».
- 2 - Klikk lagre. Då aktiverer du automatisk lagring.
- 3 - Start å skrive kode

Pass på at du har svart på alle oppgåver før du prøver å få oppgåver perfekte. Viss du meiner noko er uklart i oppgåva, presiser føresetnadene dine i svaret.

Oppgåvene byrjar på neste side, og er i tillegg delt ut på papir.

Oppgave 4 (15% ~ 36 minutt) – Trådar og trådsikkerheit

Du skal lage eit program som opprettar to trådar. Kvar tråd skal utføre ei løkke som aukar verdien til ein felles teljevariabel eit tal gonger. T.d. viss teljevariabelen startar på 0, og løkka har 10 000 gjennomløp, så skal/bør teljaren ha verdien 20 000 når begge trådane er ferdige. Du kan bruke ein `static int teller` som ein «global» delt teljevariabel. Programmet skal skrive ut på skjermen den endelege verdien i teljaren etter at begge trådane er ferdige.

Du kan velje sjølv korleis du definerer trådklassen(e) og opprettar trådobjekta. Her er det fleire alternativ, både via Thread og Runnable, og både via namngitt(e) og anonym(e) klassesdefinisjon(er).

Først skal du lage ein versjon som fører til ein kappløpstilstand (race condition), og deretter ein versjon der problemet er retta opp. Heile oppgåva skal leverast i eitt vedlegg, med tydeleg merka deloppgåver.

a) Kappløpstilstand (ca. 10 %)

Skriv ein versjon av programmet beskriven over, inkl. definisjon av den/dei nødvendige trådklassar, der vi har ein kappløpstilstand (race condition). Svar i tillegg på følgjande spørsmål:

- i) Kva prøver programmet å gjere? Forklar kort.
- ii) Kva kan gå gale, og kvifor?
- iii) Kvar i koden skjer feilen(e)?

b) Retta kode (ca. 5 %)

Skriv ein versjon av programmet der svakheita(ne)/feilen(e) er retta ved hjelp av synkronisering, og svar i tillegg på følgjande spørsmål:

- i) Kva løysing brukte du? Forklar kort.
- ii) Kvar i koden brukte du de(n)?
- iii) Kvifor fungerer det?

Oppgave 5 (15% ~ 36 minutt) – JavaScript programmering

Ein webapplikasjon skal verte laga for å organisera ei liste med venar. Det er inga lagring av data på webtenar.

Websida til applikasjonen inneheld to felt. Det første feltet lar bruker registrera ein ven med namn og alder. Skjermbiletet under viser korleis dette feltet kan sjå ut:

Registrer venn

Navn:

Alder (0-110):

Skjermbiletet 1: Felt for å registrera ven

Ingenting skal skje om bruker klikkar *Registrer* knappen dersom:

- Eit av felta manglar verdi.
- Alder har ein ulovleg verdi. Alder må være eit heiltal i intervallet 0 til 110.

Når knappen *Registrer* verte klikka skal venane verte vist sortert på alder slik det neste skjermbiletet, Skjermbiletet 2 viser:

Mine venner i 2025	
Alder	Navn
22	Jo Josen
23	Hanne Hannesen
37	Grete Gretesen
44	Per Persen

Skjermbiletet 2: Registrerte venar

Observer at lista med venar skal verte vist sortert på alder. Skjermbiletet 2 viser venen frå Skjermbiletet 1 plassert riktig i lista. Årstalet som verte vist øvst er året for dagen i dag.

Feltet i Skjermbilete 1 for å registrera ein ven er laga med HTML-koden som er vist i Kode døme 1 nedanfor:

```
<fieldset id="registrering">
  <legend>Registrer venn</legend>

  <label for="navn">Navn:</label>
  <input type="text" size="20" id="navn">
  <label for="alder">Alder (0-110):</label>
  <input type="number" id="alder" min="0" max="110">
  <button type="button">Registrer</button>
</fieldset>
```

Kode døme 1: HTML-kode for å registrera ven

Feltet i Skjermbilete 2 for å vise lista med venar er laga med HTML-koden i Kode døme 2 nedanfor:

```
<fieldset id="visning">
  <legend>Mine venner i <span></span></legend>
  <table>
    <thead><tr><th>Alder</th><th>Navn</th></tr></thead>
    <tbody></tbody>
  </table>
</fieldset>
```

Kode døme 2: HTML-kode for å vise liste med venar

Applikasjonen inneholdt følgende JavaScript-kode:

```
class Vennkontroller {
  #navnefelt;
  #alderfelt;
  // Fyll inn evt. flere felt

  constructor(felt1, felt2) {
    // Kode for felt1, felt for å registrere venn
    this.#navnefelt = felt1.getElementsByTagName("input")[0];
    this.#alderfelt = felt1.getElementsByTagName("input")[1];
    felt1.querySelector("button").addEventListener(
      "click",
      () => { this.#registrervenn() }
    );

    // Legg inn evt. mere kode her
  }

  #registrervenn() {
    /* Legg inn kode her for å vise venn fra Skjermbilde 1 i listen
       som vises i Skjermbilde 2 */
  }
}

const felt1 = document.getElementById("registrering");
const felt2 = document.getElementById("visning");

new Vennkontroller(felt1, felt2);
```

Kode dørne 3: JavaScript-kode for å registrera og vise venar

Oppgave: Fyll inn den manglande koden i JavaScript-klassa **Vennkontroller**.

Hjelp:

- HTML input-element har i JavaScript ein eigenskap *value* som er noverande verdi til feltet.
- Om JavaScript sitt **Number**:
 - Statisk metode *parseInt(string)* konverterar frå **String** til **Number**.
 - Statisk metode *isInteger(arg)* returnerer sann om argumentet er eit heiltal, om ikkje usann.
- Om JavaScript sin **Array**:
 - Statisk metode *from(liste)* returnerer førekomst av **Array** frå ein listestruktur.

```
const HTMLTbl = document.getElementsByTagName("table")[0];
const rows = HTMLTbl.tBodies[0].rows;
const rowArray = Array.from(rows);
```

- Metode *findIndex(funksjon)* returnerer fyrste indeks i førekomst av **Array** der test-funksjonen returnerer sann. Metoden returnerer verdi -1 om test-funksjonen er usann for alle tabellelement.

```
const tabell = [1, 5, 12, 8];
const partall = (element) => element % 2 === 0;

console.log(tabell.findIndex(partall));
// Skriv ut talet 2, indeks til 12 som er fyrste partal
```

- Om JavaScript sitt **HTMLTableSectionElement** (HTML taggar *TBODY* og *THEAD*):
 - Metode *insertRow(indeks)* set inn og returnerer ei ny HTML tabellrekke (tagg *TR*) i posisjon *indeks* i elementet.

Om *indeks* har verdi -1 verte tabellrekka lagt inn på slutten av elementet.

- Om JavaScript sitt **HTMLTableRowElement** (tagg *TR*):
 - Metode *insertCell(indeks)* set inn og returnerer ei ny HTML tabellcelle (tagg *TD*) i posisjon *indeks* i tabellrekka.

Om *indeks* har verdi -1 verte tabellcella lagt inn på slutten av tabellrekka.

- HTML **Node** elementet sin eigenskap *textContent* representerer alt tekstinnhald i noden og alle barn hans.

Eigenskapen kan verte tilordna ein ny verdi og vil då sette inn nytt tekstinnhald, til dømes i ei tabellcelle.

Oppgave 6 (40% ~ 96 minutt) – Web back-end med Spring MVC

I denne oppgåva skal du skrive litt kode (Spring og JSP) for ein tenkt nettbutikk som sel buttons.

Nedanfor ser du nettsida. Øvste del viser varane som blir selde i butikken, og nedste del viser varane brukaren har lagt i handlekorga i økta/sesjonen. Viss handlekorga er tom, blir teksten «*Handlekurven er tom*» vist i staden for radene med varer.

Verkemåte:

- URL for nettsida er **/butikk**, og denne blir bygd opp frå dynamiske data for vareutval og handlekorg. Data om varer (**id**, navn, pris og bilde) blir henta frå ein database.
- Når bruker vel eit tal (antall) av en vare og trykkjer «Legg i handlekurv» (Sti: **/handle**), blir det valde talet av denne vara lagt til i handlekorga. Deretter blir sida vist på nytt med oppdatert handlekorg.
- Når bruker trykker søppelbøtte [] til høgre for vare i handlekorga (Sti: **/fjerne**), blir denne vara fjerna heilt frå handlekorga. Deretter blir sida vist på nytt med oppdatert handlekorg.
- Når bruker trykker «Gå til kassen» kan vi gå til ein tenkt sti kalla **/kasse**.

Våre populære buttons!



Give me a break

Pris: 99,-

Antall:



Hello, World!

Pris: 109,-

Antall:



I love Java

Pris: 89,-

Antall:

Din handlekurv

Varenavn	Stykkpris	Antall	Beløp	
Hello, World!	109,-	3	327,-	
I love Java	89,-	5	445,-	
Totalt			772,-	Gå til kassen

Til bruk i løysinga di tenker vi at vi har klassane **Vare**, **Ordrelinje** og **Handlekurv**.

- **Vare** representerer ei vare frå vareutvalet i databasen.
- **Ordrelinje** (ofte også kalla varelinje) er eit tal av ei vare (f.eks. 3 stk. «Hello World!»-button). Denne har også ein metode som gir beløp for eit tal av ei vare.
- **Handlekurv** lar oss legge til eit tal av ei vare, fjerne ei ordrelinje, hente ut lista av ordrelinjer. Denne har også ein metode som gir oss totalbeløp for alt i handlekorga.

Du kan rekne med at desse klassene finst og verkar heilt fint.

```
@Entity
public class Vare {

    @Id private int id;
    String navn;
    int pris;
    String bildefil;

    //Gettere m.m. ...

}
```

```
public class Ordrelinje {

    private Vare vare;
    private int antall;

    //Gettere m.m. ...

    public int getBelop() {
        return vare.getPris() * antall;
    }

}
```

```
public class Handlekurv {

    public void leggTilEtAntallAvEnVare(Vare vare, int antall) {}
    public void fjernOrdrelinje(int vareid) {}
    public boolean isTom() {}
    public Collection<Ordrelinje> getOrdrelinjer() {}
    public int getTotal() {}

}
```

Du kan også rekne med at det er definert eit interface **VareRepository** som lét deg hente ut varer frå databasen. I løysinga kan du bruke dette direkte i controlleren, dvs. det er ikkje nødvendig å putte ein service mellom controller og repository.

Krav til løysninga di:

- Løysninga må bruke Spring-rammeverket for controlleren
- Løysninga må bruke JSP, Expression Language (EL) og Standard Template Lib (JSTL) for viewet
- Løysninga må følgje Post-Redirect-Get-prinsippet (PRG)

Oppgåver

a) (10%) Skriv ein Spring-controller **ButikkController** som behandlar en førespurnad om å sjå nettsida, altså **GET /butikk**. Viss ingen handlekorg finst i sesjonen, skal det opprettast ei tom handlekorg for sesjonen. Varer blir henta på nytt frå repositoriet ved kvar førespurnad. Viewet for denne sida er **butikk**.

b) (15%) Skriv eit JSP-view **butikk.jsp** som byggjer opp nettsida ut frå dei dataane som er klargjorde. Hugs at nettsida må innehalde HTML-kode for å utføre alle brukarval som vist i illustrasjonen, dvs. leggje eit tal av ei vare i handlekorg, fjerne ordrelinje for ei vare, og gå til kassa.

Tips:

- Nettsida på biletet i oppgåveteksten bruker eit tabelloppsett for å få det ryddig. Dette treng ikkje du å gjere i svaret ditt! Det som er viktig er datainnhald og logisk struktur for å byggje ei fungerande side.
- Nettsida har ein «Legg i handlekurv»-knapp per vare og ein søppelbøtte []-knapp per ordrelinje. Ei grei løysing her kan vere å leggje opp til fleire <form>-element, og identifisere desse med vare-id.
- Du kan anta at vare.bildefil gir ein komplett filsti til biletet for vara.

c) (5%) Utvid **ButikkController** med ein metode som behandlar en førespurnad om å leggje eit tal av ei vare i handlekorga. Sti: **/handle**. Du kan rekne med at førespurnaden kjem med gyldige data frå butikksida.

d) (5%) Utvid **ButikkController** med ein metode som behandlar en førespurnad om å fjerne ei ordrelinje for ei vare frå handlekorga. Sti: **/fjerne**. Du kan rekne med at førespurnaden kjem med gyldige data frå butikksida.

e) (5%) Slik applikasjonen er no, vil brukaren starte med tom handlekorg igjen viss nettlesaren blir lukka og brukaren kjem tilbake til butikken neste dag. Beskriv korleis vi kunne «behalde» handlekorga til neste økt. Løysninga skal ikkje krevje innlogging, og skal heller ikkje lagre noko i databasen mellom øktene. For full score må du beskrive løysinga i detalj, men du treng ikkje å skrive Java-kode.